

ISLAMIC UNIVERSITY OF TECHNOLOGY



MACHINE LEARNING LAB

CSE 4622

Lab 4: Neural Networks from Scratch

Author:

Rahatut Tahrim Mounota
220041122

Contents

1	Task 1: Implementation	2
1.1	Python Code Implementation	2
2	Task 2: Comparison and Analysis	4
2.1	Influence of Hidden Layer Size (H)	4
2.2	Influence of Learning Rate (α)	5
3	Task 3: Custom Learning Rate Scheduler	8
3.1	Scheduling Execution Logic	8
3.2	Performance Metrics Summary	8
3.3	Scheduler Performance Analysis	9

1 Task 1: Implementation

The objective of this task was to construct a standard single-hidden-layer feedforward neural network entirely from scratch without high-level modeling frameworks. The network utilizes Xavier initialization, Rectified Linear Unit (ReLU) activations for hidden transformations, and a Sigmoid activation mapping the final layer output to binary state probabilities.

Optimization is handled over a standard loop via Binary Cross-Entropy (BCE) loss minimization. The baseline model was trained for 300 epochs using a learning rate of $\alpha = 0.05$ and a hidden layer configuration of $H = 16$.

The core functions were implemented manually according to the following mathematical specifications:

- **ReLU Activation:**

$$\text{ReLU}(z) = \max(0.0, z) \quad (1)$$

- **Sigmoid Activation:**

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

- **Binary Cross-Entropy Loss:**

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \quad (3)$$

1.1 Python Code Implementation

Listing 1: Core Neural Network Implementation Steps

```
def relu(z):  
    return max(0.0, z)  
  
def sigmoid(z):  
    return 1.0 / (1.0 + np.exp(-z))  
  
def forward_prop_single(x, W1, b1, W2, b2):  
    num_hidden_layer = len(W1)  
    num_features = len(x)
```

```

z1 = []
for j in range(num_hidden_layer):
    z = 0.0
    for k in range(num_features):
        z += W1[j][k] * x[k]
    z += b1[j]
    z1.append(z)

a1 = [relu(z) for z in z1]

z2 = 0.0
for j in range(num_hidden_layer):
    z2 += W2[j] * a1[j]
z2 += b2

y_hat = sigmoid(z2)
return z1, a1, z2, y_hat

```

2 Task 2: Comparison and Analysis

2.1 Influence of Hidden Layer Size (H)

The model’s dimension mapping dictates its spatial transformation limits. Evaluating the target space across configurations $H \in \{4, 8, 16, 32\}$ produced varying metrics.

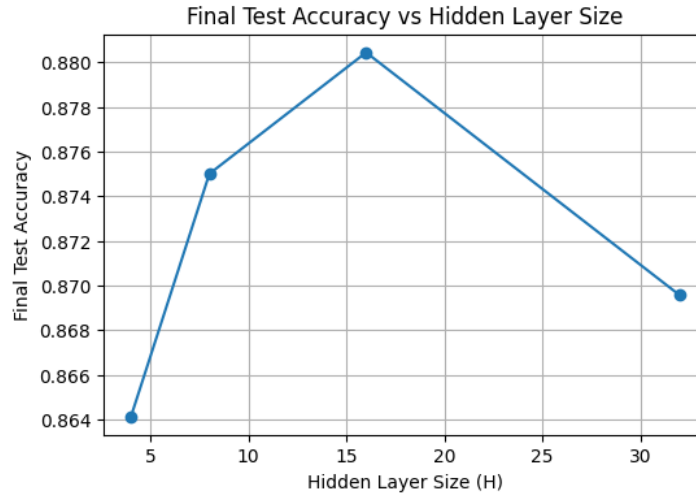


Figure 1: Final Test Accuracy vs. Hidden Layer Size (H)

Analytical Assessment

The hidden layer size H determines the network’s geometric capacity to construct decision hyperplanes from the input space. If H is too small, the network underfits due to insufficient feature expression capabilities to approximate non-linear structures.

Conversely, when H becomes extremely large, the network is prone to overfitting by mapping random variance or dataset-specific noise instead of systemic signals. Thus, scaling hidden layers upwards does not indefinitely yield better validation tracking; optimal generalization requires balancing dimensional capacity.

2.2 Influence of Learning Rate (α)

To capture gradient trajectory optimization behaviors, separate tracking sweeps were established across $\alpha \in \{0.001, 0.01, 0.05, 0.1\}$.

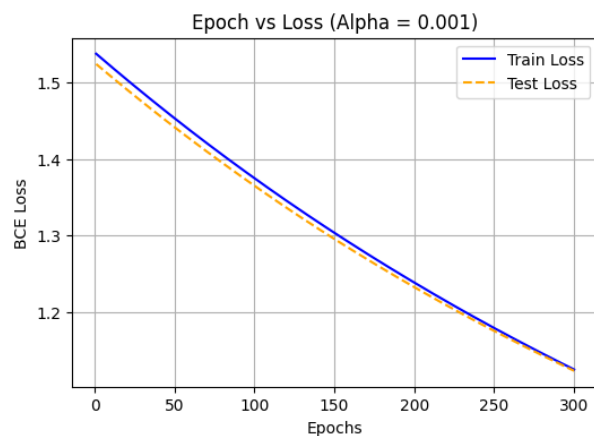


Figure 2: Epoch vs. Loss ($\alpha = 0.001$)

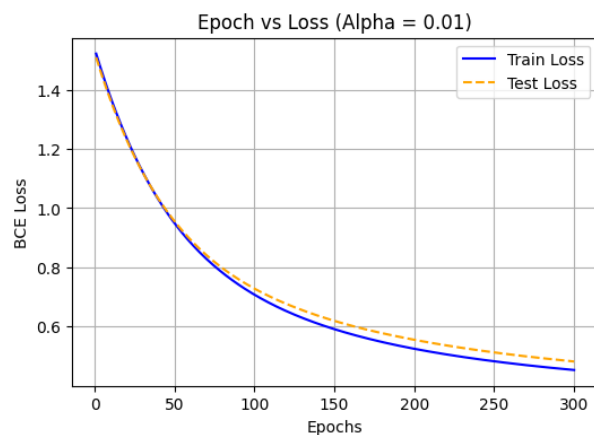


Figure 3: Epoch vs. Loss ($\alpha = 0.01$)

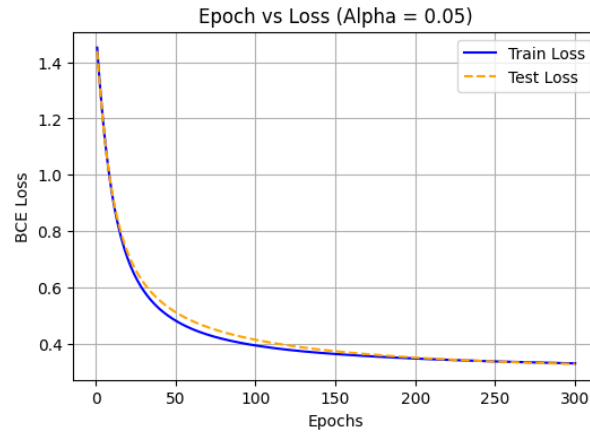


Figure 4: Epoch vs. Loss ($\alpha = 0.05$)

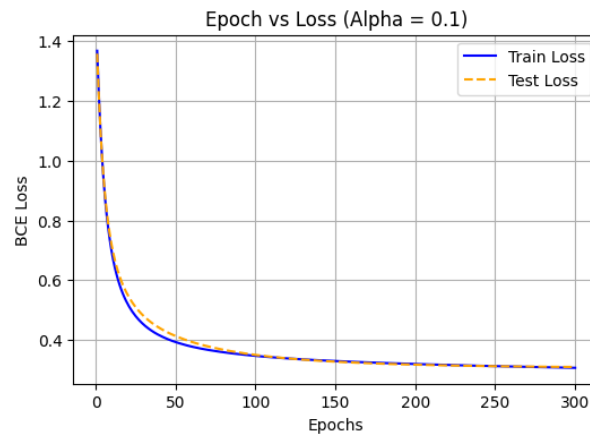


Figure 5: Epoch vs. Loss ($\alpha = 0.1$)

The relationship summarizing hyperparameter learning speeds against the final validation response performance metric can be seen below:

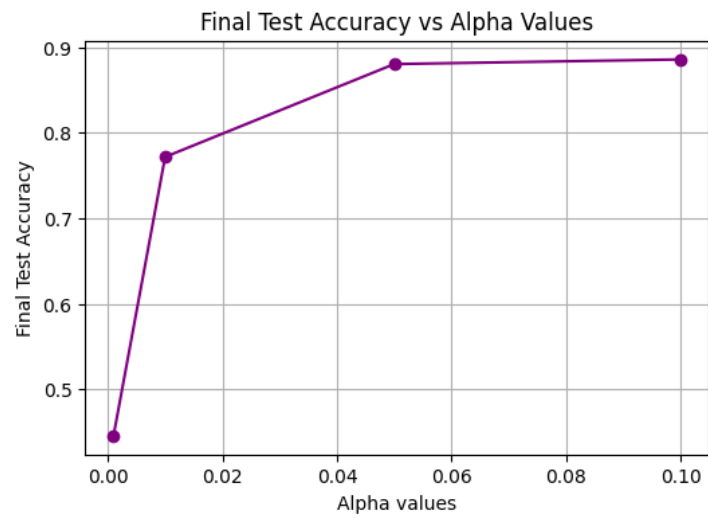


Figure 6: Final Test Accuracy vs. Alpha (α)

3 Task 3: Custom Learning Rate Scheduler

To stabilize optimization pathways, two distinct custom step-update scheduling mechanics were integrated directly into the core gradient loop routines:

- **Step Decay Scheduler:** Reduces the parameter search scaling velocity factor by exactly 1/10th sequentially upon crossing every 100 epoch milestone bounds.
- **Triangular Scheduler:** Executes an active warm-up cycle ramping linearly up between an interval window of $[0.001, 0.1]$ up until the midway point (Epoch 150), subsequently mirroring a structural cooling compression curve tracking back down linearly to baseline boundaries.

3.1 Scheduling Execution Logic

Listing 2: Learning Rate Adjustment Mechanism

```
# Step Decay Scheduler Engine
if scheduler_type == "step":
    if epoch > 0 and epoch % 100 == 0:
        current_lr = current_lr * 0.1

# Triangular Scheduler Engine
elif scheduler_type == "triangular":
    midpoint = epochs // 2
    if epoch <= midpoint:
        current_lr = 0.001 + (0.1 - 0.001) * (epoch /
            midpoint)
    else:
        current_lr = 0.1 - (0.1 - 0.001) * ((epoch -
            midpoint) / midpoint)
```

3.2 Performance Metrics Summary

The comparative metric behaviors tracked across optimization paradigms are compiled within the index table below:

Table 1: Optimization Mechanics Performance Comparison Summary

Scheduler Configuration Profile	Final Observed Test Accuracy
Step Decay Optimization Profile	0.8424
Triangular Cyclic Variant Profile	0.8804

3.3 Scheduler Performance Analysis

The **Step Decay Scheduler** offers distinct tracking benefits by permitting massive optimization steps during early chaotic states, before scaling down to fine-tune weight updates near local minima without overshooting.

The **Triangular Scheduler** excels further by leveraging a deliberate gradient "warm-up" phase. This design safely guides parameters out of poor initial error states before cooling, which maximizes optimization stability and yields higher evaluation accuracy.